

ADQAPI & .NET User's Guide

OVERVIEW

This document explains how to interface the ADQAPI through our .NET wrapper. The .NET wrapper consists of two DLLs generated by SWIG (www.swig.org) and may be used with e.g. Visual Basic or C#. These files are installed with the Windows installer. With the help of this wrapper, the programmer does not have to manually wrap every single function that s/he wants to use.

1 STRUCTURE

All functions and constants are called through a .NET class, defined in ADQAPI_dotNet_helper.dll. These in turn call the ADQAPI_dotNet.dll that talks to the ADQAPI.

For example, this call in C:

```
ADQAPI_GetRevision()
```

Is translated to this .NET call:

```
ADQAPI_dotNet.ADQAPI_GetRevision()
```

The file ADQAPI_dotNet_helper.dll must be referenced into the Visual Studio project before compilation. The example *ADQAPI_VisualBasic_example*, provided with the installer, is set up to use the DLL from C:\Windows\system32.

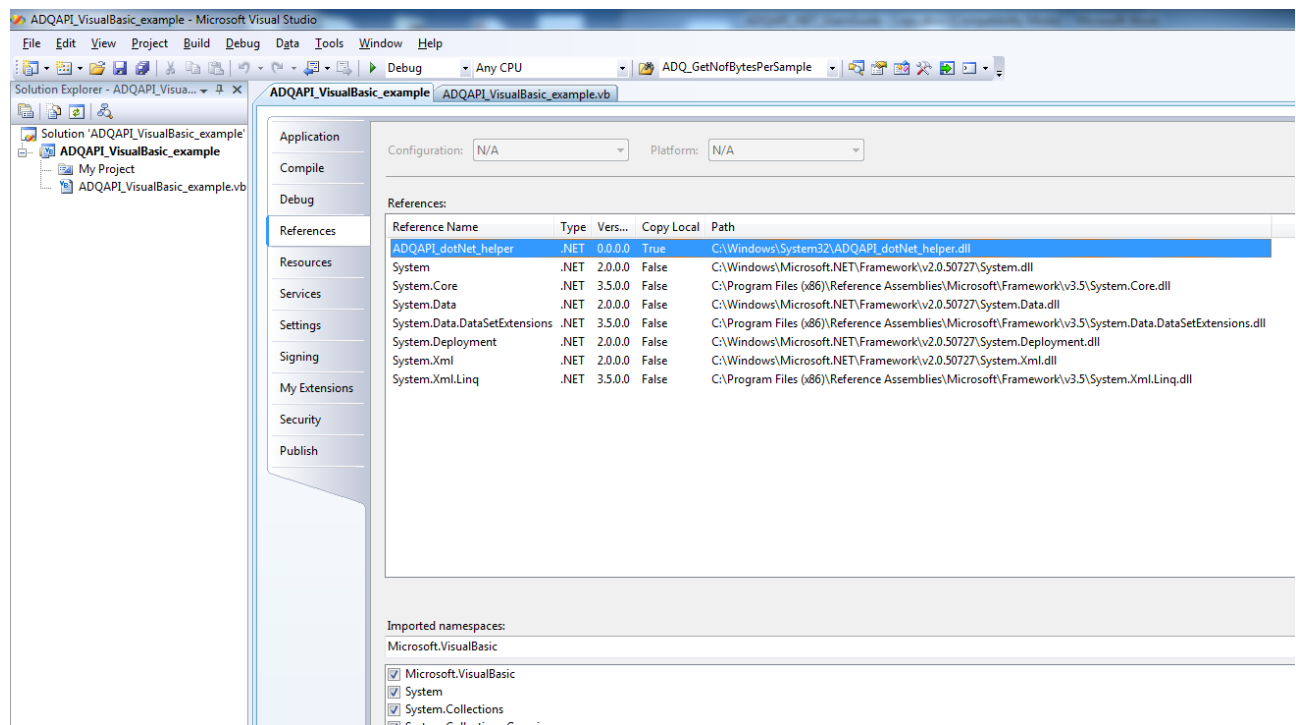


Figure 1: The reference path may be changed under ADQAPI_VisualBasic_example->Properties->References

2 HOW TO HANDLE POINTERS

Because the ADQAPI is a C-API, the .NET interface uses pointers. These are implemented as .NET objects by SWIG. SWIG also generates functions for creating arrays, pointers and pointers to pointers, as well as functions for casting to and from void pointers. Some understanding of C data types and pointers is helpful for the programmer. The *ADQAPI_VisualBasic_example* implements one of the more tricky cases of



handling C-pointers with the .NET wrapper (when allocating buffers to transfer data with the ADQ_GetData function).

3 THE VISUAL BASIC EXAMPLE

Our Visual Basic example has been written to highlight some of the details of interfacing ADQAPI through the .NET wrapper. The example sets up a basic MultiRecord collection, triggers the unit and collects the data. It is based on “ADQAPI_simple_example”, written in C. Thus, the document 13-1323_ADQAPI_simple_example_UsersGuide may be used to better understand the flow of the program.

Translating the program into any language within .NET should be straight-forward, as the API-calls themselves are implemented as a .NET object and therefore are the same between languages.

Note: In general we do not recommend using Visual Basic for any application that has any kind of requirement on transfer speed, as the data transfer rate achievable in Visual Basic is very low.

3.1 Compiling and running

After installing the software from SP Devices, our Visual Basic example may be found in the install directory, typically (but not always) “C:\Program Files\SP Devices\ADQAPI_VisualBasic_examples\ADQAPI_VisualBasic_example.zip”. Unpack the .zip file to a folder of your choosing. Then open the ADQAPI_VisualBasic_example.sln solution file. Depending on your Visual Studio version, the project file may have to first be converted. Visual Studio generally does this well, so proceed. When the project is opened, you can connect your unit and build and run the example.

The example will generate a few output .asc files (one for each record). This data format is not very space effective, but will let you see the output in a readable way. The files may also be drag-and-dropped into ADCaptureLab for convenient plotting.

By default, the example project has a reference to ADQAPI_dotNet_helper.dll within C:\Windows\system32. If running the program results in an error when interfacing the DLL, please verify that this reference is valid.